

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 9

Gestión del estado con Provider y ChangeNotifier

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles

SEMESTRE 6.º · Mención Ing. de Software

TEMA Estado compartido con Provider

DURACIÓN 1 sesión de laboratorio

CAPÍTULO Cap. 4 — Manejo del estado con Provider

MODALIDAD Laboratorio · individual

DOCENTE M. Sc. Lic. Huáscar Fedor Gonzales Guzmán

REQUISITO Guías de Laboratorio N.º 1 a N.º 5

NOTA · Inicia el Capítulo 4

Hasta ahora manejaste el estado con `setState`, que solo alcanza al widget donde vive. Cuando varias pantallas necesitan **el mismo dato** (un carrito, la sesión, un tema), pasarlo de una a otra se vuelve incómodo. **Provider** resuelve esto: guarda el estado en un lugar y deja que cualquier pantalla lo lea o lo modifique.

01 Objetivos

Objetivo general. Gestionar estado compartido entre varias pantallas usando el paquete Provider con ChangeNotifier, construyendo un carrito de compras accesible desde toda la app.

Objetivos específicos:

- Agregar y configurar la dependencia `provider` en el proyecto.
- Crear un modelo de estado con `ChangeNotifier` y `notifyListeners()`.
- Exponer el estado a la app con `ChangeNotifierProvider`.
- Leer y modificar el estado con `context.watch`, `context.read` y `Consumer`.
- Compartir el carrito entre la pantalla de productos y la del carrito.

02 Marco teórico

DEFINICIÓN · El problema que resuelve Provider

`setState` actualiza solo su propio widget. Para compartir estado había que pasarlo por los constructores pantalla por pantalla ("prop drilling"). **Provider** coloca el estado por encima del árbol de widgets, y cualquier descendiente lo obtiene sin recibirlo por parámetro.

DEFINICIÓN · ChangeNotifier y notifyListeners

Un `ChangeNotifier` es una clase que guarda estado y avisa cuando cambia. Tras modificar sus datos llama a `notifyListeners()`, y todos los widgets que lo escuchan se reconstruyen automáticamente.

DEFINICIÓN · watch, read y Consumer

`context.watch<T>()`: lee y se reconstruye cuando el estado cambia (úsalo en `build`).

`context.read<T>()`: lee una vez sin escuchar (úsalo en callbacks, como `onPressed`). `Consumer<T>`: reconstruye solo una parte del árbol.

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 a N.º 5 (en especial listas, navegación y estado con `setState`).

Conceptos que se reutilizan:

`ListView.builder`

`Navigator.push`

Modelo de datos

NOTA · De setState a Provider

No olvides `setState`: sigue siendo lo correcto para estado local de un solo widget (un campo, una animación). **Provider** es para el estado que varias pantallas comparten. Saber cuándo usar cada uno es parte del objetivo.

04 Desarrollo · Parte A — Dependencia y modelo de estado

1 Agregar el paquete provider

1. Crea el proyecto `carrito_app`. Agrega la dependencia con la terminal en la raíz del proyecto:

POWERSHELL
`flutter pub add provider`

2. Esto añade la línea a `pubspec.yaml`. Verifica que quede algo así (la versión puede variar):

PUBSPEC.YAML · YAML
dependencies:
 flutter:
 sdk: flutter
 provider: ^6.1.2

NOTA · flutter pub add vs. editar a mano

`flutter pub add provider` agrega la última versión compatible y ejecuta `flutter pub get` por ti. También puedes escribir la línea a mano y luego correr `flutter pub get`.

2 Modelar el producto y el carrito

1. En `lib/main.dart`, define un modelo simple de producto y unos datos de ejemplo:

LIB/MAIN.DART · DART

```
class Producto {
  final String nombre;
  final double precio;

  const Producto(this.nombre, this.precio);
}

const catalogo = <Producto>[
  Producto('Cuaderno', 15.0),
  Producto('Boligrafo', 5.0),
  Producto('Mochila', 120.0),
  Producto('Calculadora', 90.0),
];
```

3 Crear el ChangeNotifier del carrito

1. Ahora la pieza central: una clase `CarritoModel` que extiende `ChangeNotifier` y expone operaciones que avisan de los cambios:

LIB/MAIN.DART · DART

```
class CarritoModel extends ChangeNotifier {
  final List<Producto> _items = [];

  List<Producto> get items => List.unmodifiable(_items);
  int get cantidad => _items.length;
  double get total =>
    _items.fold(0, (suma, p) => suma + p.precio);

  void agregar(Producto p) {
    _items.add(p);
    notifyListeners();
  }

  void quitar(Producto p) {
    _items.remove(p);
    notifyListeners();
  }

  void vaciar() {
    _items.clear();
    notifyListeners();
  }
}
```

ADVERTENCIA · No olvides notifyListeners()

Cada método que cambie los datos debe terminar con `notifyListeners()`. Si se te olvida, el estado cambia pero la interfaz **no se entera** y no se actualiza.

05 Desarrollo · Parte B — Exponer y consumir el estado

4 Proveer el estado en la raíz

1. Envuelve la app en un `ChangeNotifierProvider` para que todas las pantallas accedan al carrito:

LIB/MAIN.DART · DART

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CarritoModel(),
      child: const MiApp(),
    ),
  );
}
```

NOTA · ¿Por qué en la raíz?

Al proveer el `CarritoModel` por encima de `MiApp`, queda disponible para toda la app: tanto la pantalla de productos como la del carrito comparten la **misma** instancia.

5 Pantalla de productos que agrega al carrito

1. Lista el catálogo. El botón usa `context.read` (no necesita escuchar). El `AppBar` muestra la cantidad con `context.watch`:

LIB/MAIN.DART · DART

```
class PantallaProductos extends StatelessWidget {
  const PantallaProductos({super.key});

  @override
  Widget build(BuildContext context) {
    final cantidad = context.watch<CarritoModel>().cantidad;
    return Scaffold(
      appBar: AppBar(
        title: Text('Productos (' + cantidad.toString() + ')'),
        actions: [
          IconButton(
            icon: const Icon(Icons.shopping_cart),
            onPressed: () => Navigator.push(
              context,
              MaterialPageRoute(
                builder: (_) => const PantallaCarrito(),
              ),
            ),
          ),
        ],
      ),
      body: ListView.builder(
        itemCount: catalogo.length,
```

```

itemBuilder: (context, index) {
  final p = catalogo[index];
  return ListTile(
    title: Text(p.nombre),
    subtitle: Text('Bs ' + p.precio.toStringAsFixed(2)),
    trailing: IconButton(
      icon: const Icon(Icons.add),
      onPressed: () =>
        context.read<CarritoModel>().agregar(p),
    ),
  );
},
),
);
}
}

```

ADVERTENCIA · read en callbacks, watch en build

En `onPressed` usamos `read` porque solo queremos **ejecutar** una acción. Usar `watch` dentro de un callback es un error común: `watch` es para reconstruir la UI, no para llamadas puntuales.

6 Pantalla del carrito con Consumer

1. La segunda pantalla lee el mismo carrito. Aquí usamos `Consumer` para reconstruir la lista y el total cuando cambian:

LIB/MAIN.DART · DART

```

class PantallaCarrito extends StatelessWidget {
  const PantallaCarrito({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Carrito')),
      body: Consumer<CarritoModel>(
        builder: (context, carrito, child) {
          if (carrito.items.isEmpty) {
            return const Center(child: Text('Carrito vacio'));
          }
          return Column(
            children: [
              Expanded(
                child: ListView.builder(
                  itemCount: carrito.items.length,
                  itemBuilder: (context, index) {
                    final p = carrito.items[index];
                    return ListTile(
                      title: Text(p.nombre),
                      subtitle: Text('Bs ' +

```

```

        p.precio.toStringAsFixed(2)),
        trailing: IconButton(
          icon: const Icon(Icons.delete),
          onPressed: () =>
            carrito.quitar(p),
        ),
      );
    },
  ),
),
ListTile(
  title: const Text('Total'),
  trailing: Text('Bs ' +
    carrito.total.toStringAsFixed(2)),
),
],
);
},
),
);
}
}

```

2. Ejecuta: agrega productos desde la primera pantalla, abre el carrito y observa cómo la cantidad, la lista y el total se mantienen sincronizados entre ambas pantallas.

06 setState vs. Provider

Ambos son válidos; la clave es el alcance del estado:

Aspecto	setState	Provider
Alcance	Un solo widget	Toda la app o subárbol
Compartir estado	Por constructores	Sin pasar parámetros
Ideal para	Campo, animación local	Carrito, sesión, tema

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste la práctica:

- ☐ La dependencia provider está en pubspec.yaml y el proyecto compila.
- ☐ CarritoModel extiende ChangeNotifier y llama a notifyListeners().
- ☐ La app está envuelta en ChangeNotifierProvider.
- ☐ Los productos se agregan con context.read en el callback.
- ☐ El AppBar refleja la cantidad con context.watch.
- ☐ La pantalla del carrito muestra ítems y total sincronizados.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura de la pantalla de productos con la cantidad en el AppBar; (2) captura del carrito con varios ítems y el total; y (3) **reto personal:** agrega la **cantidad por producto** (que un mismo ítem sume unidades en vez de repetirse) y un botón "Vaciar carrito" usando el método `vaciar()`. Descríbelo en 3–5 líneas.

08 Cuestionario

1. ¿Qué problema de `setState` resuelve Provider cuando el estado debe compartirse?
2. ¿Qué hace `notifyListeners()` y qué ocurre si se olvida llamarlo?
3. ¿Cuál es la diferencia entre `context.watch` y `context.read` y cuándo usar cada uno?
4. ¿Para qué sirve `ChangeNotifierProvider` y por qué se coloca en la raíz?
5. ¿En qué casos seguirías usando `setState` en lugar de Provider?

09 Rúbrica de evaluación

Criterio	Puntaje
Dependencia provider y CarritoModel (<code>ChangeNotifier</code>)	25 %
<code>ChangeNotifierProvider</code> en la raíz	15 %
Uso correcto de <code>watch/read/Consumer</code>	30 %
Estado compartido entre las dos pantallas	15 %
Cuestionario resuelto	15 %

10 Referencias

- Flutter — Simple app state management (Provider). [docs.flutter.dev · state-mgmt/simple](https://docs.flutter.dev/state-mgmt/simple)
- Paquete provider en pub.dev. pub.dev/packages/provider
- Flutter — `ChangeNotifier` (API). [api.flutter.dev · ChangeNotifier](https://api.flutter.dev/ChangeNotifier)

Contenido basado en la documentación oficial de Flutter (v3.44) y el paquete provider (^6.x). Elaborado para INF 662 — UATF.